

ROS2

DAY 4 – Manual 1

ROS2 Services – Fundamentals & Setup

Objective

By the end of this manual, you will be able to:

1. Understand ROS2 services and their synchronous nature.
 2. Know the difference between Topics and Services.
 3. Create a CMake package for .srv files (service_pkg).
 4. Create a Python package for server and client nodes (my_py_pkg).
 5. Understand .srv file structure (TestService.srv).
 6. Build both packages and generate Python service interfaces
-

Step 0 – Pre-Requisites

- ROS2 installed (Foxy, Humble, Rolling) and python 3 installed
 - ROS2 workspace (ros2_ws) created
 - Basic understanding of ROS2 nodes and topics (from Day 1 & 2)
-

Step 1 – Understand ROS2 Services

1.1 What is a Service?

- Services allow a client node to send a request to a server node and wait for a response.
- Unlike topics, services are synchronous.

Example:

CLIENT → "Hello server"

SERVER → "hello client... Got your req"

1.2 Service vs Topic

Feature	Topic	Service
Communication	One-way stream	Two-way request/response
Frequency	Continuous	Only when requested
Example	Sensor data	Commands, calculations

1.3 Anatomy of a .srv File

- .srv files define request and response structure.
- Example: TestService.srv

```
# Request
string input
---
# Response
string output
```

Step 2 – Create CMake Package for .srv

1. Navigate to your workspace source folder:

```
cd ~/ros2_ws/src
```

2. Create service_pkg (already present):

```
ros2 pkg create --build-type ament_cmake service_pkg
```

3. Folder structure:

```
service_pkg/
  CMakeLists.txt
  package.xml
  srv/
    TestService.srv
  src/
```

Step 3 – Update CMakeLists.txt

```
find_package(ament_cmake REQUIRED)
```

```
find_package(rcpp REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "srv/TestService.srv"
  DEPENDENCIES builtin_interfaces
)

ament_export_dependencies(rosidl_default_runtime)

ament_package()
```

Step 4 – Create Python Package for Nodes

1. Navigate to `src/`:

```
cd ~/ros2_ws/src
```

2. Create `my_py_pkg`:

```
ros2 pkg create --build-type ament_python my_py_pkg
```

3. Folder structure:

```
my_py_pkg/
  my_py_pkg/
    __init__.py
    srv_server.py
    srv_client.py
  package.xml
  setup.py
  setup.cfg
  test/
```

Step 5 – Add .srv Dependency in Python Package

In `my_py_pkg/package.xml`:

```
<build_depend>service_pkg</build_depend>
<exec_depend>service_pkg</exec_depend>
```

- This allows Python nodes to import the service:

```
from service_pkg.srv import TestService
```

Step 6 – Build Workspace

1. Build `service_pkg` first (generates `.srv` interface):

```
cd ~/ros2_ws  
colcon build --packages-select service_pkg  
source install/setup.bash
```

2. Then build `my_py_pkg`:

```
colcon build --packages-select my_py_pkg  
source install/setup.bash
```

✓ Note: Build order is critical; Python nodes rely on CMake package for `.srv`.

Step 7 – Verify Setup

1. List packages:

```
ros2 pkg list | grep my_py_pkg  
ros2 pkg list | grep service_pkg
```

2. Check services (no server running yet):

```
ros2 service list
```

Step 8 – Concepts Recap

- Services are synchronous, request/response based.
- Server node waits for requests; client sends requests.
- `.srv` file defines request (input) and response (output).
- CMake package generates Python interfaces for `.srv`.
- Python package implements server/client nodes (`srv_server.py` and `srv_client.py`).

✓ Manual 1 Complete!

Next: Manual 2 will cover line-by-line server/client implementation, running, and testing.
